

Wireshark Network Analysis

Network Security — Packet Capture and Protocol Analysis

1. What Did I Do

This exercise involved installing Wireshark, capturing live network traffic on a home network, and analyzing the resulting packets using Wireshark's built-in statistical and inspection tools. All captures were performed on a personally owned home network, so no organizational permission was required.

Installation and Setup

Wireshark was downloaded from [wireshark.org](https://www.wireshark.org) and installed on a Windows PC. During installation, the WinPcap/Npcap packet capture driver was also installed, which enables Wireshark to place a network adapter into promiscuous mode — a mode in which the adapter captures all traffic it sees on the network segment, not just traffic addressed to that specific machine. Upon launching Wireshark, the welcome screen displayed all available network interfaces along with a live bandwidth sparkline for each. The active interface — Ethernet 2, the wired connection to the home router — showed real-time activity and was selected for capture.

Generating Traffic

To generate a diverse and representative capture, several types of network activity were deliberately initiated simultaneously:

- Streaming video on a smart TV connected to the same network, generating sustained external TCP/UDP traffic to content delivery network (CDN) servers.
- Uploading files to a local Network Attached Storage (NAS) device on the same subnet, generating high-volume internal SMB2 protocol traffic.
- General browser usage, producing DNS lookups, TLS handshakes, and HTTP/2 traffic to external servers.
- Allowing IoT devices (smart home speakers, mesh router nodes, and other connected equipment) to operate normally in the background.

The capture was run for approximately five minutes, resulting in over 99,000 packets totaling approximately 1.67 gigabytes of captured data.

How Wireshark Works

Wireshark operates by intercepting raw network frames as they pass through the selected adapter and decoding them layer by layer according to the OSI model. It reassembles protocol stacks automatically — for example, identifying that a given frame is an Ethernet frame carrying an IPv4 packet, which itself carries a TCP segment, which in turn carries SMB2 application data. Each captured packet is stored in memory and can be inspected individually. The main display is divided into three panes: the packet list (a row per packet with timestamp, source, destination, protocol, and summary), the

packet details (an expandable tree showing each protocol layer and its fields), and the raw hex dump. Wireshark applies color coding to protocols by default, making it visually easy to distinguish TCP, UDP, ARP, DNS, and other traffic types at a glance.

Analyzing the Output

Analysis was conducted primarily through Wireshark's Statistics menu rather than manual packet-by-packet inspection. The Protocol Hierarchy view provided a percentage breakdown of all protocols present in the capture, organized as a tree mirroring the protocol stack. The Conversations view listed every pair of IP addresses that exchanged traffic, showing packet counts, byte totals, and directional flow for each. The Endpoints view provided a per-IP summary of all traffic sent and received by each host. Individual packets of interest — particularly ARP exchanges and mDNS broadcasts — were expanded in the packet details pane to read their full contents. Display filters (such as filtering by IP address or protocol name) were applied to isolate specific traffic types for focused examination.

2. What Are the Results

Protocol Hierarchy

The Protocol Hierarchy view revealed that the capture was dominated by a single activity. Of the 1.67 GB captured, 99.4% of bytes were carried by IPv4, and within that, 98.6% of all packets were TCP. Drilling further into the TCP subtree, NetBIOS Session Service accounted for 3.9% of packets, and beneath it, SMB2 (Server Message Block Protocol version 2) accounted for 3.9% of packets but an extraordinary 99.4% of total captured bytes — approximately 1.65 GB. This represents the file upload to the NAS device. The SMB2 layer also contained a small number of DCE/RPC frames, which handle service negotiation and authentication at the start of the SMB session.

Other notable protocols in the hierarchy included Transport Layer Security (449 packets), indicating encrypted external connections consistent with streaming or background application traffic. UDP traffic included TP-Link Smart Home Protocol packets from mesh router nodes, Simple Service Discovery Protocol (SSDP) from multiple IoT devices, Multicast DNS (mDNS) from smart home devices, and standard DNS. At the Ethernet layer, a small number of IEEE 802.1Q Virtual LAN (VLAN) tagged frames were also observed, consistent with VLAN segmentation in the home mesh network.

Conversations and Endpoints

The Conversations view identified the most significant exchange as the pair X.X.X.X (the local PC) and X.X.X.X (the NAS device), which accounted for 96,734 packets and approximately 2 GB of bidirectional data — the bulk file transfer. The second most active external pairing was X.X.X.X communicating with X.X.X.X, an IP address registered to Amazon Web Services (AWS), exchanging 755 packets and 2 MB. This is consistent with streaming traffic routed through a CDN. Several other small conversations were observed with various external IPs in the 3.x, 18.x, 23.x, 34.x, 40.x,

104.x, 142.x, 172.x, 185.x, and 199.x ranges, representing background telemetry, push notifications, and service check-ins from applications and devices.

The Endpoints view confirmed an inventory of all active network participants during the capture window. Local subnet devices active during the capture included the PC at .62, the NAS at .69, the router at .1, and multiple other devices at .53, .54, .60, .61, .72, .75, .76, .78, and .80 — a combination of TP-Link Deco mesh nodes and Google/Nest smart home devices. Multicast and broadcast addresses (X.X.X.X, X.X.X.X, X.X.X.X, X.X.X.X, X.X.X.X) were also listed as endpoints, reflecting the volume of SSDP and mDNS traffic on the network.

ARP and mDNS Observations

Several ARP (Address Resolution Protocol) packets were visible in the raw capture. These included standard gateway resolution requests — for example, a device at X.X.X.X broadcasting "Who has X.X.X.X?" to obtain the router's MAC address, and the router in turn resolving the PC's MAC address. One ARP exchange specifically identified the USB-C ethernet adapter (manufactured by Ugreen) by its MAC address prefix, illustrating how ARP traffic passively exposes hardware vendor information to anyone observing the network.

The mDNS traffic was particularly voluminous. Even without any deliberate interaction with smart home devices, a sustained stream of mDNS NOTIFY and query packets was observed from Google/Nest devices on the network. These packets advertised services including Chromecast, Google Cast Group, Nest Audio, Google Nest Hub, and Googlezone over the local link-local multicast address X.X.X.X. Each advertisement broadcast the device's local hostname, service type, port, and associated IP address in plaintext. This activity occurred entirely passively — no user interaction with the devices was required to generate it.

Evidence of Cyber Attacks

No packets in the capture appeared to reflect an active cyber attack. Traffic patterns were consistent with normal home network activity: a large internal file transfer, moderate external streaming traffic, routine DNS and ARP resolution, and background IoT device communication. There were no port scans, no unusual connection attempts to non-standard ports, no malformed packets indicative of fuzzing or exploitation, and no traffic to known malicious IP ranges. The SSDP and mDNS traffic was high-volume but structurally normal.

Attack Surface Analysis

While no active attack was detected, the capture revealed several meaningful areas of attack surface — the sum of all points through which an unauthorized actor could attempt to gain access, extract information, or disrupt operations:

- **SMB2 Internal File Transfer:** The NAS upload generated 1.65 GB of SMB2 traffic on the local subnet. SMB2 without mandatory signing is vulnerable to relay attacks (NTLM relay), where an attacker on the same network intercepts

authentication exchanges and uses them to authenticate to other services. Even with encryption, the sheer volume and timing of the transfer is visible in metadata, which could inform an attacker about when and how much data is being moved.

- **IoT Device Enumeration via mDNS and SSDP:** Every Google/Nest device on the network continuously broadcast its identity, hostname, capabilities, and IP address in plaintext. An attacker with passive access to the network — for instance, through a compromised device — could build a complete inventory of smart home equipment without ever sending a single packet. This enumeration could be used to target known vulnerabilities in specific device firmware versions.
- **Hardware Vendor Exposure via ARP:** ARP traffic revealed MAC address prefixes, which directly map to hardware manufacturers (TP-Link, Ugreen/Realtek). This information assists in fingerprinting the network and selecting targeted exploits against known vendor software stacks.
- **Unencrypted Broadcast Protocols:** SSDP, mDNS, and ARP all operate in plaintext with no authentication. Any device on the local network can read, spoof, or poison these exchanges. ARP poisoning in particular could enable a man-in-the-middle attack, allowing an attacker to intercept traffic between any two local devices — including the PC and NAS file transfer.
- **External Traffic Metadata:** Even though external connections used TLS (and thus had encrypted payloads), the IP addresses communicated with, the timing of connections, and packet sizes are all visible. This metadata can reveal which services are in use, when a user is active, and patterns in behavior that could be exploited through social engineering or targeted phishing.

3. What Did I Learn

Digital Networks and Packets

This exercise provided a concrete illustration of how layered network protocols actually behave in practice. The OSI model is often taught as an abstract concept, but seeing a single file transfer represented as millions of Ethernet frames — each one carrying an IPv4 header, a TCP header, a NetBIOS header, an SMB2 header, and finally a small chunk of file data — makes the layering tangible. It also reinforced the concept of overhead: the raw data being transferred was the file, but a significant portion of the captured bytes consisted of headers, acknowledgments, retransmissions, and control messages that exist purely to manage the reliability of the delivery.

The capture also illustrated the difference between unicast, multicast, and broadcast traffic — concepts that are easy to understand in theory but more meaningful when seen together in a live capture. The NAS file transfer was point-to-point unicast. The Google devices were sending to multicast groups (X.X.X.X for mDNS, X.X.X.X for SSDP). ARP requests were sent to the broadcast address. All three types appeared simultaneously in a single five-minute window, demonstrating how diverse a seemingly quiet home network actually is.

Attack Surface as a Practical Concept

The most important takeaway from this exercise was a shift in how I think about attack surface. Before performing this capture, the home network felt relatively secure — it uses WPA3 encryption on Wi-Fi, a Pi-hole for DNS filtering, and VLAN segmentation. But the capture revealed that a significant amount of sensitive operational detail is visible to anyone already inside the network perimeter. The smart home devices alone broadcast enough information to fingerprint every piece of hardware, its software services, and its location on the network — all in plaintext, all without prompting.

This reinforces a key principle in cybersecurity: perimeter defense is necessary but not sufficient. Once an attacker has any foothold inside a network — whether through a phished credential, a vulnerable IoT device firmware, or a compromised guest device — passive observation alone provides an enormous amount of intelligence. Defense in depth strategies such as network segmentation (VLANs isolating IoT devices from the primary LAN), SMB signing requirements, and monitoring for anomalous multicast traffic become essential layers of protection.

Future Applications

Wireshark is directly applicable to real-world cybersecurity work in several ways. For incident response, packet captures taken at the time of a suspected breach provide forensic evidence of what traffic occurred, to and from which hosts, and what protocols were in use. For network hardening, baseline captures like this one identify unexpected services and protocols that can be disabled or firewalled. For threat hunting, filters can be applied to search for indicators of compromise — for example, traffic to known malicious IPs, unusual port usage, or signs of protocol anomalies consistent with exploitation.

From an organizational standpoint, regular packet capture analysis — even brief spot checks — can surface rogue devices, unauthorized data exfiltration, and misconfigured services that would otherwise be invisible. In a larger enterprise environment, full packet capture is typically integrated with a Security Information and Event Management (SIEM) system, where automated correlation rules can flag anomalies in real time. The manual analysis skills developed in this exercise are the foundation for understanding what those automated systems are detecting and why.

This exercise also demonstrated the value of generating varied traffic intentionally when testing or validating network visibility. By combining internal file transfers, external streaming, and passive IoT activity, the capture illustrated how different types of network use produce distinctly different traffic signatures — a skill that is directly useful when profiling network behavior as part of a security audit or network design review.